华东师范大学

曙光高性能集群七八期

用户手册

法律条款

1. 对本文档的任何使用均被视为完全理解并接受本文档列举的所有法律条款。
2. 本文档的所有权利归作者所有，作者保留所有权利。
3. 若本文档的保密级别为“公开发布”，允许学术使用，未经作者书面授权，禁止商业使用。若本文档的保密级别为“内部资料”，仅授权曙光信息产业（北京）有限公司的在职员工使用。若本文档的保密级别为“核心资料”，未经作者书面授权，禁止任何形式的使用。
4. 对本文档的使用形式包括但不限于存储、出版、复制、传播、展示、引用、编辑。使用过程中不得对本文档作任何增减编辑，引用时需注明出处。商业使用是指在商业活动中或有商业目的活动中的使用。学术使用是指以技术交流或学术研究为目的的使用。
5. 实施任何侵权行为的法人或自然人都必须向作者支付赔偿金，赔偿金计算方法为：

   赔偿金 = 涉案人次×涉案时长（天）×涉案文档份数×受众人次×基数×1 元人民币，

   涉案人次、涉案时长、涉案文档份数、受众人次小于 1 时，按 1 计算。若保密级别为“公开发布”，基数为 100；若保密级别为“内部资料”，基数为 1000；若保密级别为“核心资料”，基数为 10000.
6. 对举报侵权行为、提供有价值证据的自然人或法人，作者承诺奖励案件实际赔偿金的 50%.
7. 作者不保证文档内容的正确性。对文档的使用后果，作者不承担任何责任。
8. 涉及本文档的法律纠纷由作者所在地法院裁决。
9. 本文档所列举法律条款的最终解释权归作者所有。

目录

1 7、8 期集群信息.....1
1.1 7 期集群.....1
1.1.1 集群资源信息.....1
1.1.2 集群系统拓扑图.....1
1.1.3 集群配套运行环境.....2
1.1.4 Module 环境.....2
1.1.5 集群配套运行环境.....3
1.2 8 期集群.....3
1.2.1 集群资源信息.....3
1.2.2 集群系统拓扑图.....4
1.2.3 集群配套运行环境.....4
1.2.4 Module 环境.....5
1.2.5 集群配套运行环境.....6
2 使用集群进行计算业务.....6
2.1 7/8 期集群账号申请使用.....6
2.2 系统登陆.....7
2.2.1 登陆 IP 地址.....7
2.2.2 命令行终端登录.....7
2.2.3 文件上传下载.....14
2.3 编译、安装 OpenMPI 示例.....16
2.4 命令行运行程序.....17
2.4.1 运行串行程序.....17
2.4.2 使用 openmpi 运行并行程序.....17
2.4.3 使用 mpich 运行并行程序.....18
2.4.4 使用 mvapich2 运行并行程序.....19
2.4.5 使用 intelmpi 运行并行程序.....19
2.5 使用作业调度（slurm 命令行模式）运行程序.....21
2.5.1 slurm 的常用命令.....21

2.5.2 作业提交命令.....27
2.5.3 作业调度系统使用举例.....32
注意事项.....32
2.6 SLURM 作业提交步骤.....32
3 软件安装及使用规范.....33
4 关机和启动.....33
5 集群管理人员联系方式.....34
6 使用问题反馈.....34

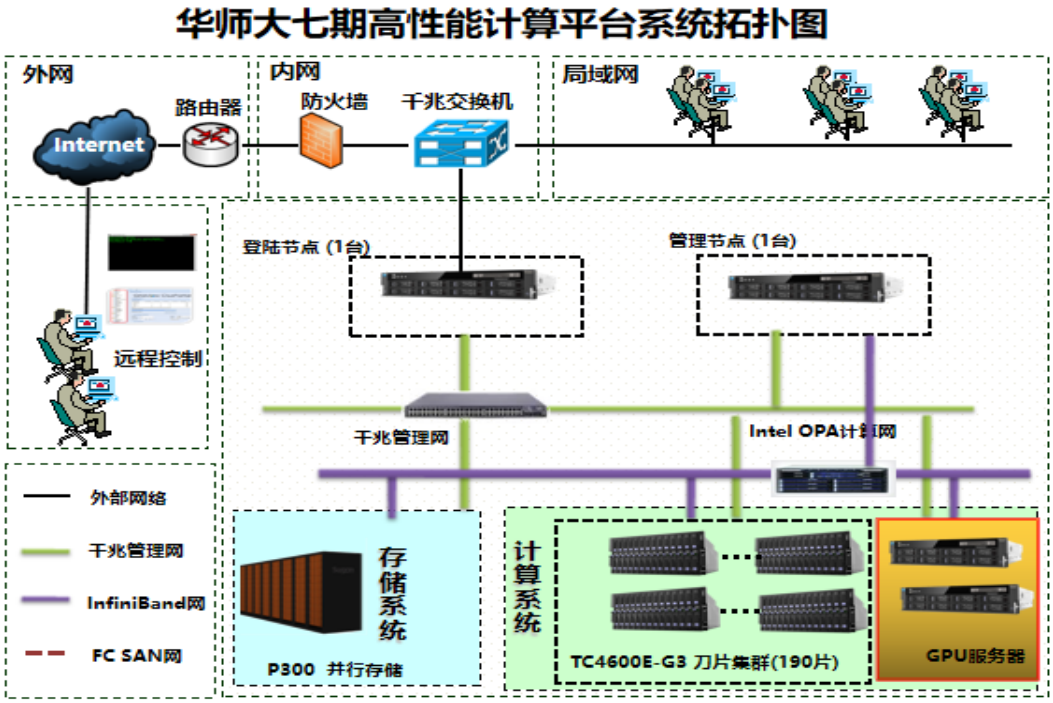
1 7、8 期集群信息

1.1 7 期集群

1.1.1 集群资源信息

- CPU 计算资源： 190 个双路计算刀片，每台配置 2*Intel 6130 2.1GHz 16 核 CPU ，128GB 内存；
- GPU 计算资源：10 台双路 GPU 服务器，每台配置 2*Intel 6240R 24 核心 CPU，384GB 内存，2* Nvidia A100-40GB 计算卡，共计 20 块 A100 nv 卡资源；
- 存储配置：1 套曙光 Parastor300 分布式并行存储系统，存储系统裸容量 500TB，采用纠删码式数据保护方式，有效空间利用率 67%，可用空间 310TB
- 集群配置 1 台管理节点，作为集群调度/管理使用。
- 1 台登录访问节点，作为用户访问使用；
- 计算、存储网络采用 100Gb/s 的 Intel OPA 网络，作为集群 MPI 计算和分布式存储 IO 读写的高速网络。
- 管理网络采用千兆网络，配置带内和带外管理。方便远程管理和操作。

1.1.2 集群系统拓扑图



1.1.3 集群配套运行环境

- 操作系统

  CentOS Linux release 7.4.1708

- 集群调度管理软件

  Gridview HPC 版本

- 编译器

  INTEL 2017

  GNU 4.8.5

  MPI

  OpenMPI 2.1.2

  INTEL MPI 2017

  MVAPICH2

- 函数库

  MKL　2017.5.239

  fftw3　3.3.7

  lapack 3.4.2

- CUDA 库

  cuda 11.0/11.3

1.1.4 Module 环境

```
[root@gpu8 ~]# module av

------------------------------------------------- /public/software/modules/ -------------------------
apps/vasp/intelmpi/5.4.1              benchmark/mpigraph/intelmpi/1.4        mathlib/netcdf/intel/4.1.3
benchmark/clusbench/1.4               benchmark/stream/intel/5.10            mpi/intelmpi/2017.4.239
benchmark/hpcg/intelmpi/2017          compiler/intel/intel-compiler-2017.5.239 mpi/mvapich2/gcc/2.3b_hfi
benchmark/hpl/intelmpi/2.2            mathlib/fftw/intelmpi/3.3.7_double     mpi/mvapich2/intel/2.3b_hfi
benchmark/i7z/gnu/0.28                mathlib/fftw/intelmpi/3.3.7_float      mpi/openmpi/gcc/2.1.2_hfi
benchmark/imb/intelmpi/2017           mathlib/hdf5/intel/1.8.12              mpi/openmpi/intel/2.1.2_hfi
benchmark/iozone/gnu/3.430            mathlib/lapack/intel/3.4.2
[root@gpu8 ~]#
```

1.1.5 集群配套运行环境

编译环境			
序号	软件名	版本	安装路径
1	GNU Compiler	4.8.5	/usr/
2	Intel Compiler	composer_xe_2017	/public/software/compiler/intel/composer_xe_2017.5.239/
MPI环境			
序号	软件名	版本	安装路径
1	intelmpi	2017	/public/software/mpi/intelmpi/2017.4.239
2	openmpi	2.1.2	/public/software/mpi/openmpi/2.1.2/intel
数学库			
序号	软件名	版本	安装路径
1	MKL	2017.5.239	/public/software/compiler/intel/composer_xe_2017.5.239/mkl
2	fftw3-float	3.3.7	/public/software/mathlib/fftw/3.3.7/float/
3	fftw3-double	3.3.7	/public/software/mathlib/fftw/3.3.7/double/
4	lapack-gnu	3.4.2	/public/software/mathlib/lapack/3.4.2/gnu
5	lapack-intel	3.4.2	/public/software/mathlib/lapack/3.4.2/intel
CUDA环境			
1	Cuda	11.0	/public/software/compiler/cuda-11.0
2	Cuda	11.3	/public/software/compiler/cuda-11.3

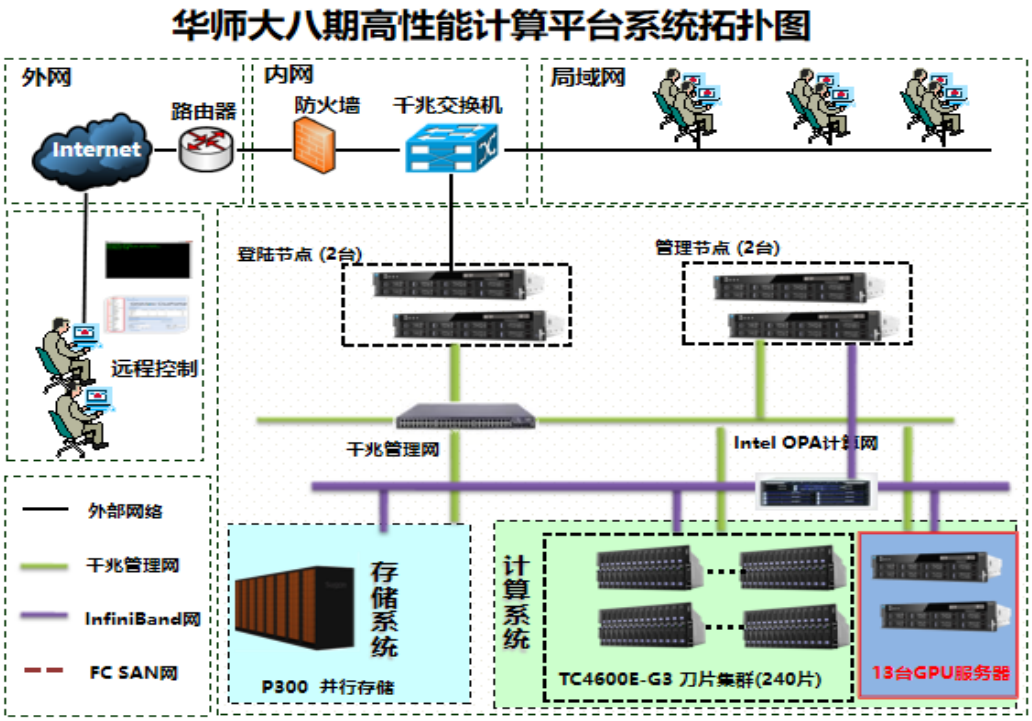
1.2 8 期集群

1.2.1 集群资源信息

- CPU 计算资源：240 台计算刀片，每台配置 2*Intel6132　14 核 CPU，内存配置 200 台为 96GB，40 台为 192GB；
- GPU 计算资源，共计 21 台双路 GPU 服务器，每台配置为 2 颗 Intel 6240R 24 核心 CPU， 384GB 内存： 其中 8 台服务器每台服务器配置 2 块 Nvidia 的 v100-16G 计算卡;11 台服务器配置 2 块 Nvidia 的 A100-40GB 计算卡；另外 2 台 4 块 Nvidia 的 A100-40GB 计算卡；集群共计 16 块 v100,30 块 A100 nv 卡计算资源；
- 集群配置 2 台管理节点，作为集群调度/管理使用；
- 集群配置 2 台登录节点，作为用户访问使用；
- 2 套并行存储，计算存储 ParaStor300，采用纠删码式数据保护方式，裸容量 528TB，有效空间利用率 67% 。备份云存储 ParaStor300 ，裸容量 5PB,采用纠删码式数据保护方式，有效空间利用率 80%；

- 计算、存储网络采用 100Gb/s 的 Intel OPA 网络，作为集群 MPI 计算和分布式存储 IO 读写的高速网络；
- 管理网络采用千兆网络，配置带内和带外管理，方便远程管理和操作。

1.2.2 集群系统拓扑图



1.2.3 集群配套运行环境

- 操作系统

  CentOS Linux release 7.4.1708

- 集群调度管理软件

  Gridview HPC 版本

- 编译器

  INTEL 2017

  GNU 4.8.5

  MPI

  OpenMPI 2.1.2

  INTEL MPI 2017

  MVAPICH2

- 函数库

  MKL　2017.5.239

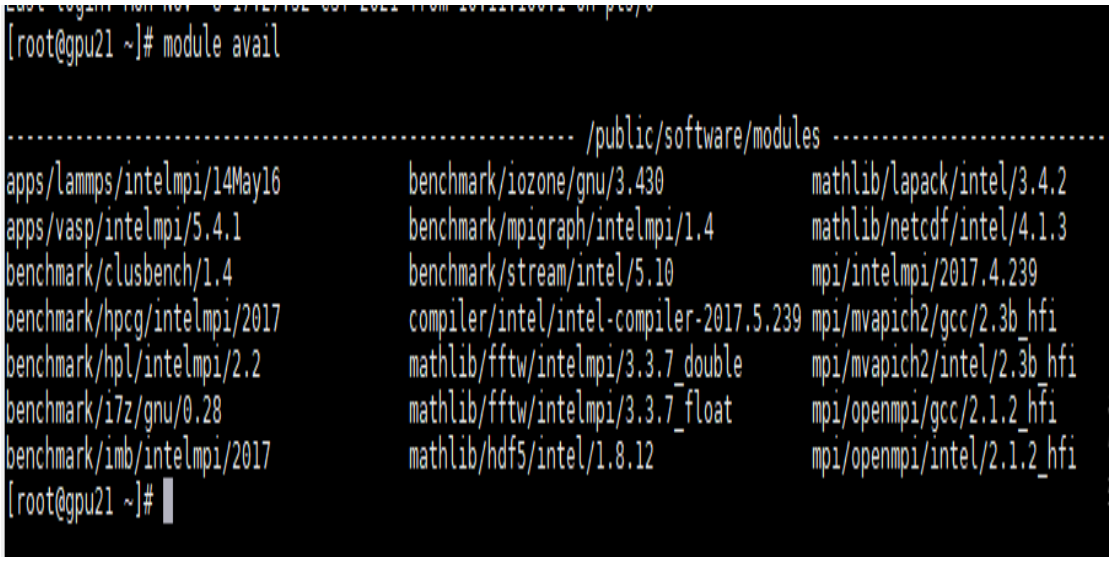
  fftw3　3.3.7

  lapack 3.4.2

- CUDA 库

  cuda 11.0/11.3/10.0

1.2.4 Module 环境



1.2.5 集群配套运行环境

编译环境			
序号	软件名	版本	安装路径
1	GNU Compiler	4.8.5	/usr/
2	Intel Compiler	composer_xe_2017	/public/software/compiler/intel/composer_xe_2017.5.239/
MPI环境			
序号	软件名	版本	安装路径
1	intelmpi	2017	/public/software/mpi/intelmpi/2017.4.239
2	openmpi	2.1.2	/public/software/mpi/openmpi/2.1.2/intel
数学库			
序号	软件名	版本	安装路径
1	MKL	2017.5.239	/public/software/compiler/intel/composer_xe_2017.5.239/mkl
2	fftw3-float	3.3.7	/public/software/mathlib/fftw/3.3.7/float/
3	fftw3-double	3.3.7	/public/software/mathlib/fftw/3.3.7/double/
4	lapack-gnu	3.4.2	/public/software/mathlib/lapack/3.4.2/gnu
5	lapack-intel	3.4.2	/public/software/mathlib/lapack/3.4.2/intel
CUDA环境			
1	Cuda	11.0	/public/software/compiler/cuda-11.0
2	Cuda	11.3	/public/software/compiler/cuda-11.3
3	Cuda	10.0	/usr/loca/cuda-10.0

2 使用集群进行计算业务

2.1 7/8 期集群账号申请使用

1. 严禁使用集群账号及计算、存储资源进行与账号所属项目无关的任何活动和行为，违者必究并将视影响、损失轻重予以锁闭账号等处理。
2. 主机资源申请，联系系统管理员进行申请。管理员联系方式见本说明维护人员联系方式。
3. 建议避免公用账号，避免项目移交的时候数据混乱。
4. 人员发生调动时，请调动人员做好数据移交，并及时通知管理员申请权限改动。
5. 临时用户项目结束或阶段性完成时，请及时通知管理员清理/锁闭/注销临时账号及项目组成员账号。
6. 用户数据和程序（非公用程序）文件一律存放于自己的属主目录下，任务计算也都是在自己的属主目录下进行、完成。每个账号开设的时候都会设置默认 5TB 的 quota，如需要更大的计算存储空间，账号申请时一并提出，由 HPC 平台管理老师根据集群存储资源情况进行审核；特别说明，集群里提供的使用空间为供用户任务计算所使用，非用户

长期数据存储空间，用户对自己的数据备份等负责，重要数据用户自行定期备份。

7. 个人账号只限个人使用，严禁将帐号和密码泄露给外单位和项目组外人员，公共组账号由管理员负责控制和使用分发。

8. 密码应定期更换，建议 8 位以上并注意加强密码复杂度。

2.2 系统登陆

2.2.1 登陆 IP 地址

用户使用高性能计算集群，一般要登陆集群的登陆节点 login，华东师范大学 7 期的高性能计算平台登陆的 IP 地址分别为 59.78.189.140，59.78.189.141，登陆访问的协议为 SSH，登陆端口为 22；

华东师范大学 HPC 8 期平台的登陆服务器为 2 台，登陆地址分别为 **59.78.189.132**，59.78.189.133，登陆访问的协议为 SSH，登陆端口也都是 2323 用户可以通过多种方式登录。

2.2.2 命令行终端登录

Windows 用户可以用 SSH Secure Shell Client，MobaXterm，SecureCRT 等 SSH 客户端软件登录。推荐使用 SSH Secure Shell Client，它集成了 SFTP 文件上传下载功能。

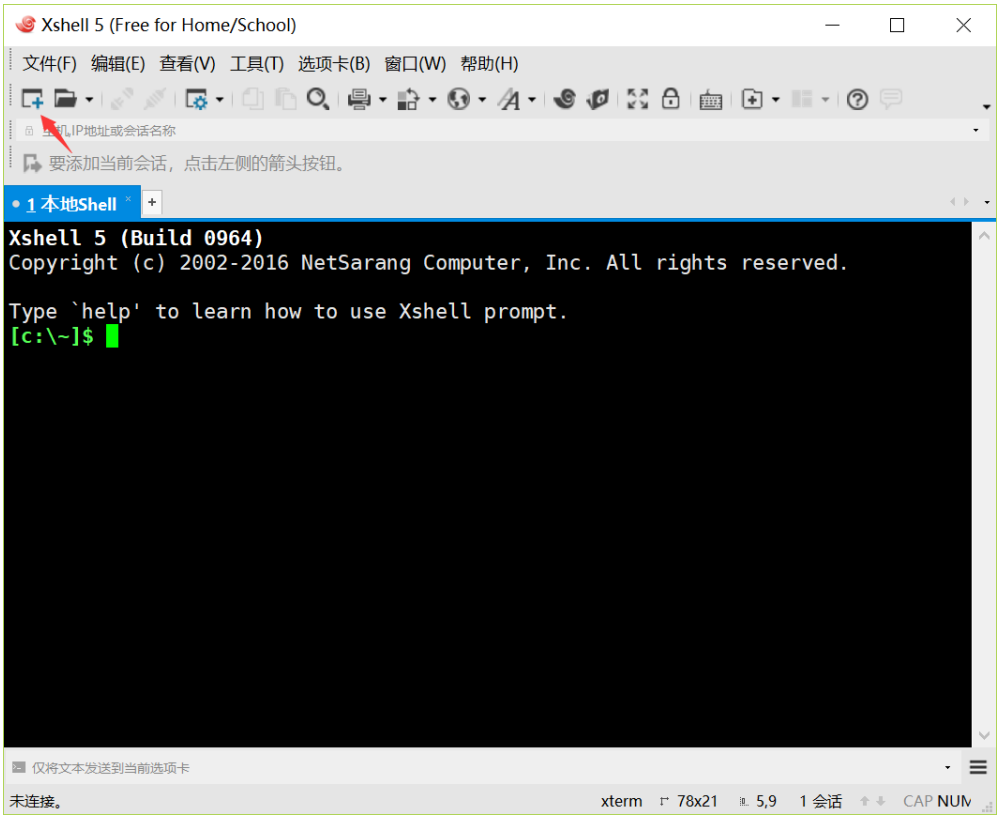
Linux 用户可以直接在命令行终端中执行 ssh 命令进行登录,并按提示输入自己的密码：

```
$ ssh username@59.78.189.141 -p22
```

下面以 xshell 为例介绍登陆方法

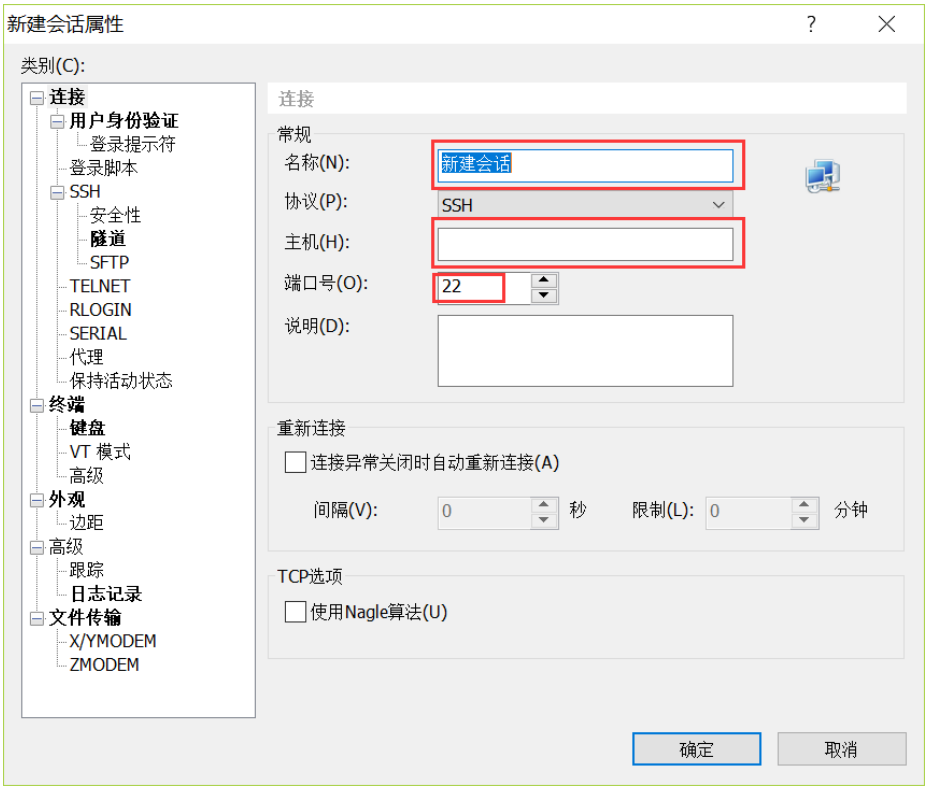
1：首先选择新建一个会话

会话用来保存要登陆的 HPC 集群 IP 地址（或域名），SSH 端口号，用户名和密码（或密钥）信息，保存为 Profile 后，后续可直接点击保存好的 Profile 登陆，不需要每次都输入用户名和密码。



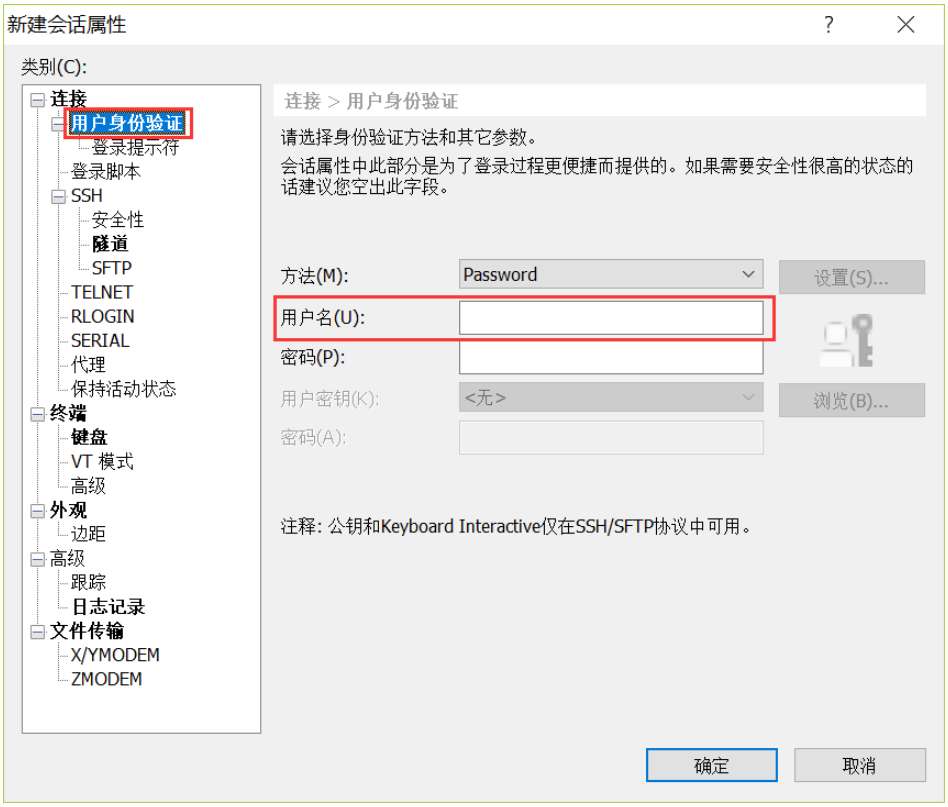
2. 编辑会话，填写登陆节点 IP 地址

在下图中填写会话名称（可任意填写），主机部分添加高性能集群登陆节点的 IP 地址或域名，ssh 登陆端口一般选择默认端口 22.

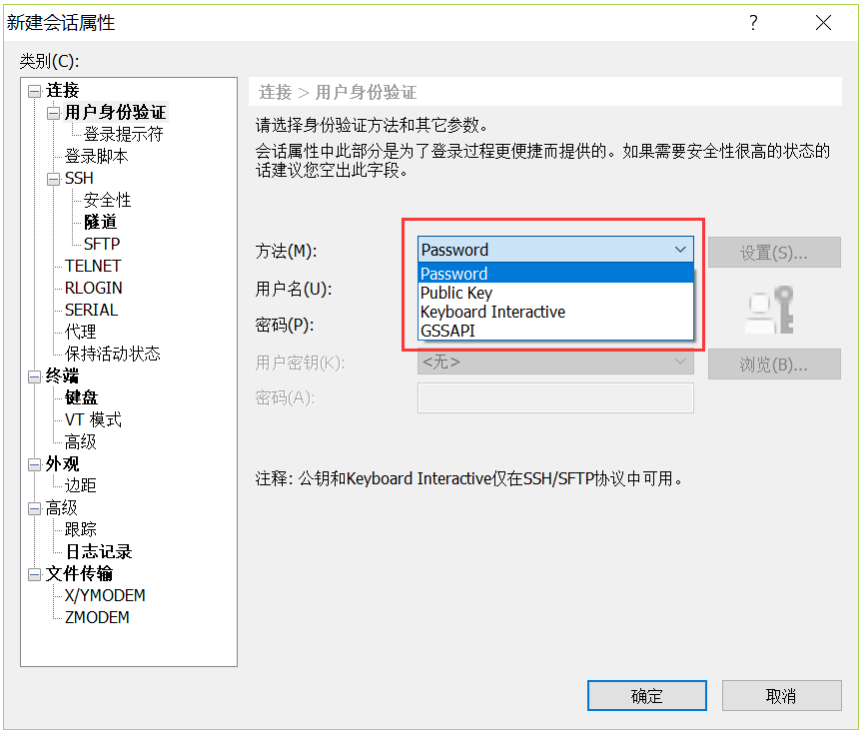


3. 填写用户名和密码（或密钥）

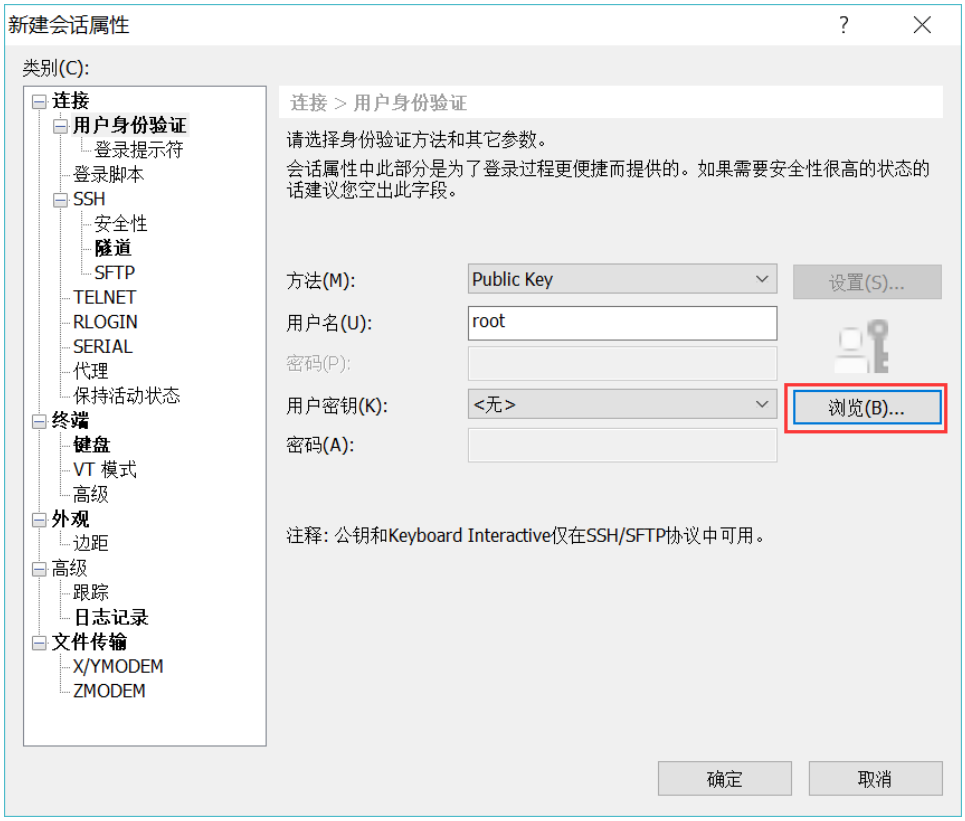
在该部分填写要登陆的用户密码或密钥。如使用密码登陆，可直接在下图中填写用户名及密码。

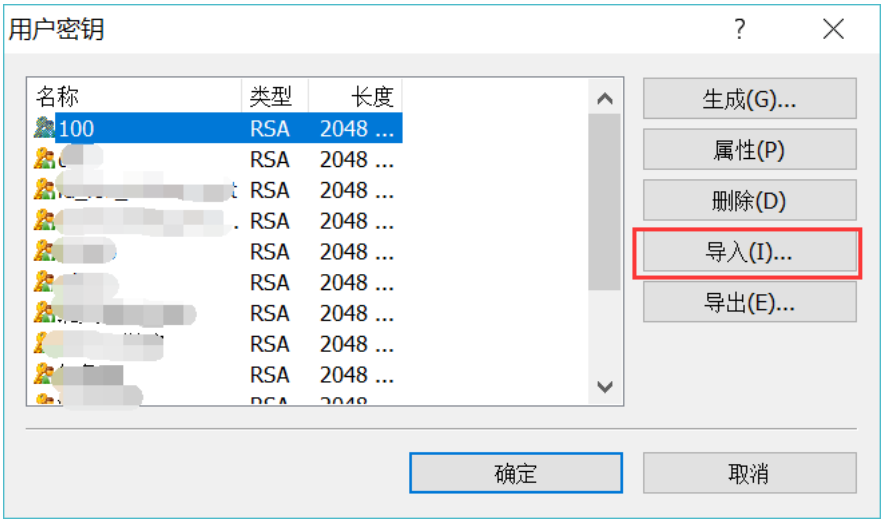


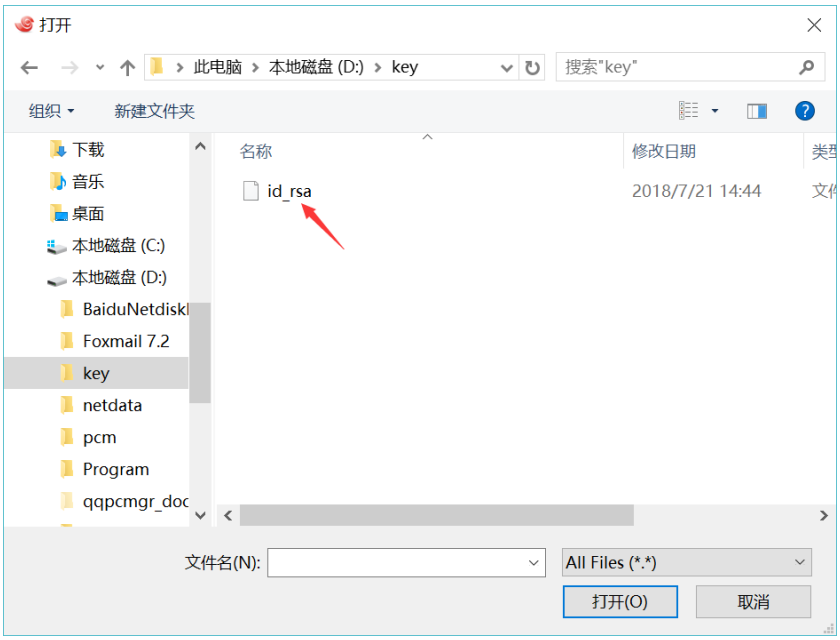
如果使用密钥登陆，可在方法中切换为 Public key 方式

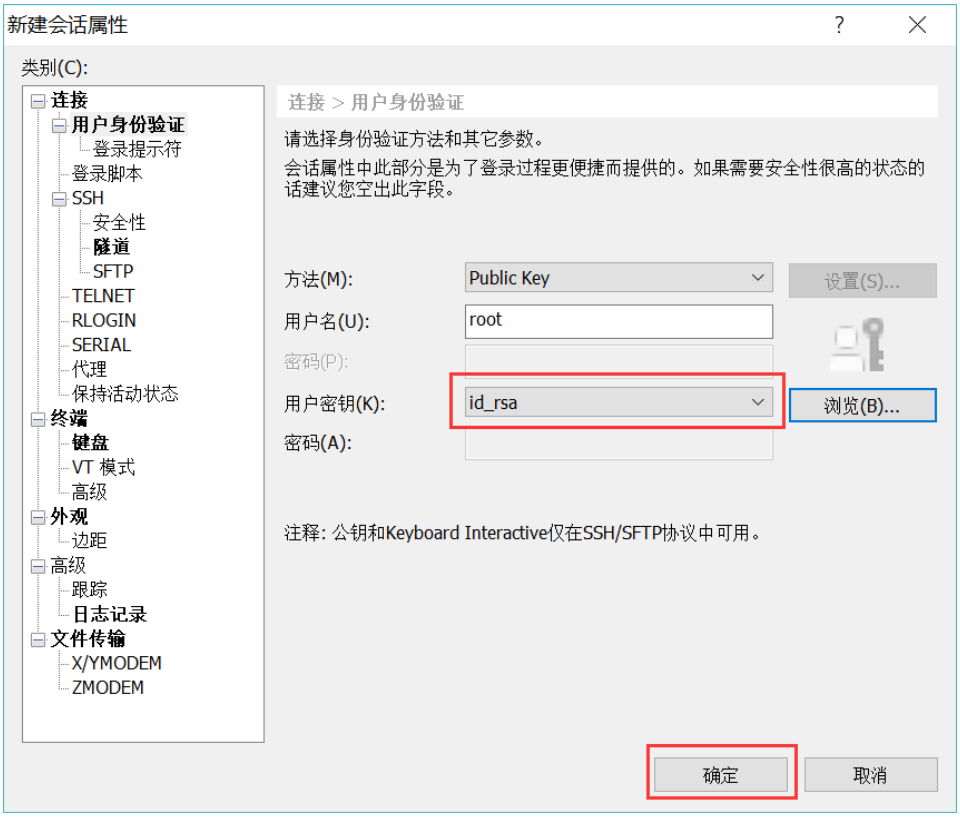


点击用户密钥旁的浏览，导入密钥

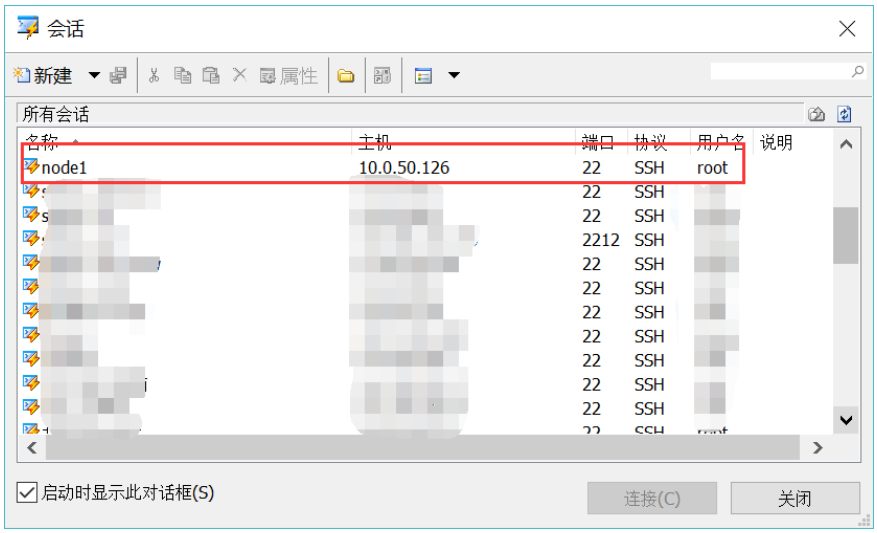

新建会话属性
类别(C):
连接
用户身份验证
登录提示符
登录脚本
SSH
安全性
隧道
SFTP
TELNET
RLOGIN
SERIAL
代理
保持活动状态
终端
键盘
VT 模式
高级
外观
边距
高级
跟踪
日志记录
文件传输
X/YMODEM
ZMODEM
连接 > 用户身份验证
请选择身份验证方法和其它参数。
会话属性中此部分是为了登录过程更便捷而提供的。如果需要安全性很高的状态的话建议您空出此字段。
方法(M):
Public Key
设置(S)...
用户名(U):
root
密码(P):
用户密钥(K):
<无>
浏览(B)...
密码(A):
注释: 公钥和Keyboard Interactive仅在SSH/SFTP协议中可用。
确定
取消


用户密钥
名称
类型
长度
100
RSA
2048 ...
生成(G)...
属性(P)
删除(D)
导入(I)...
导出(E)...
确定
取消

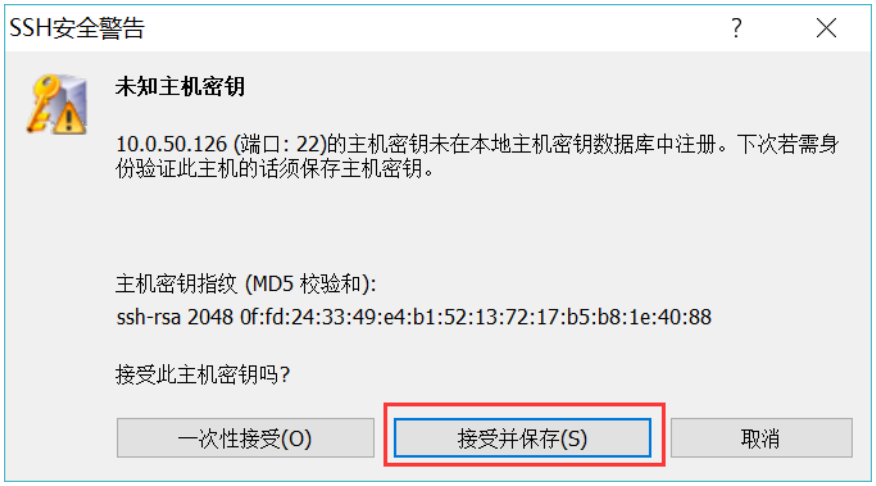




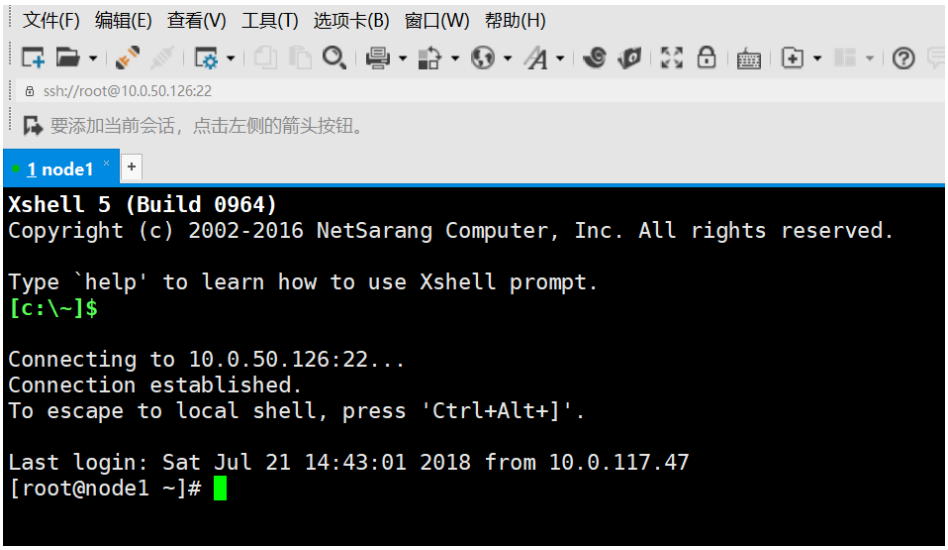
3. 选择创建完成的会话，完成登陆



首次登陆时会弹出窗口，询问是否保存主机密钥，选择保存



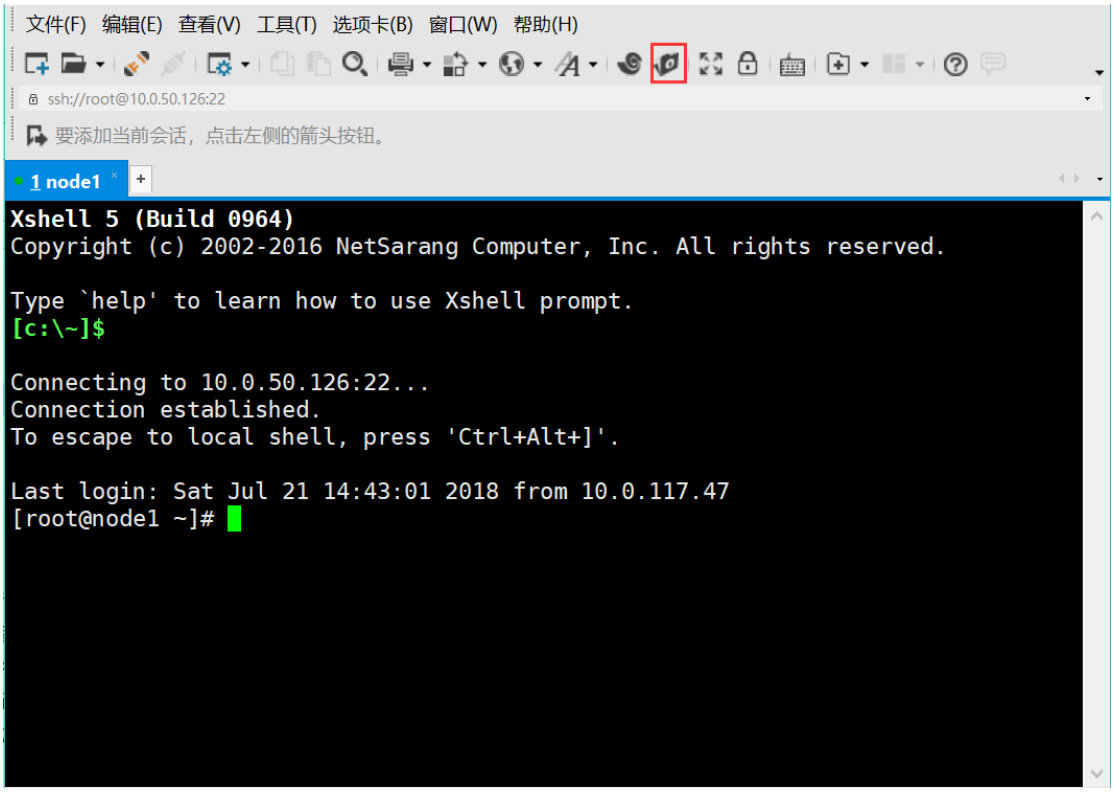
登陆完成，可以使用命令行界面



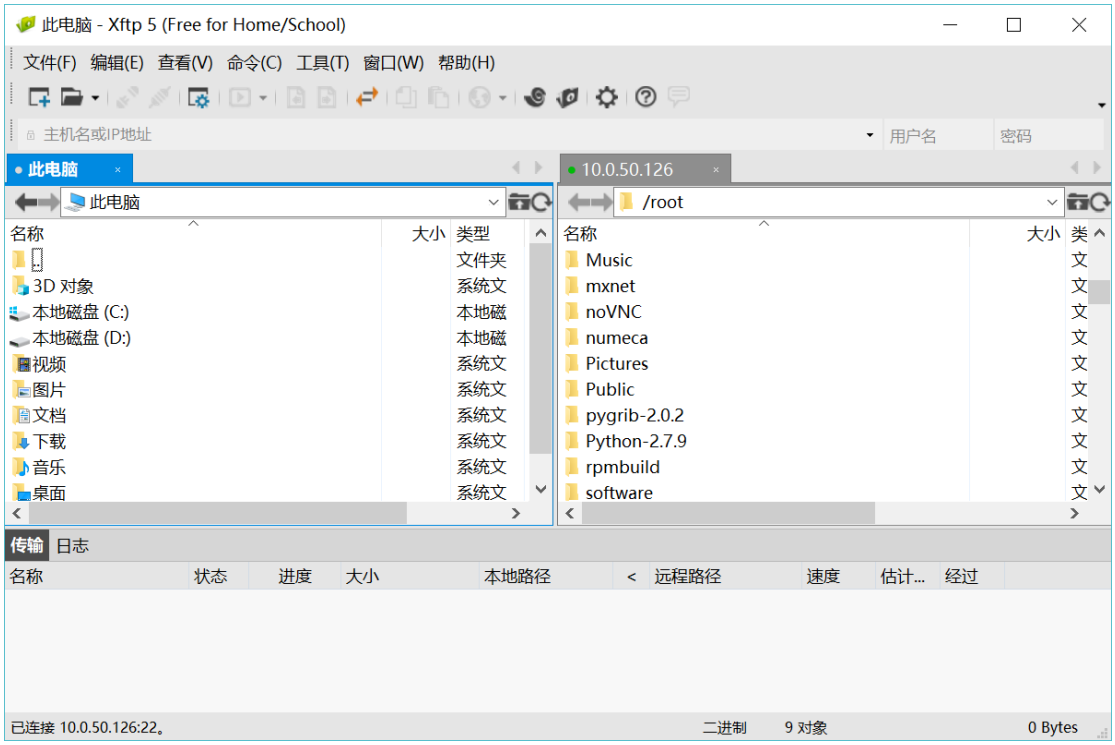
2.2.3 文件上传下载

Windows 用户可以用 SSH Secure Shell Client，winscp, Xftp 等软件实现文件的上传下载。

下面以 Xftp 为例，介绍文件上传和下载使用方法。Xftp 与 Xshell 进行了集成，使用 xshell 登陆到集群后，点击工具栏上文件夹的图标，可以打开 Xftp 文件传输的窗口。

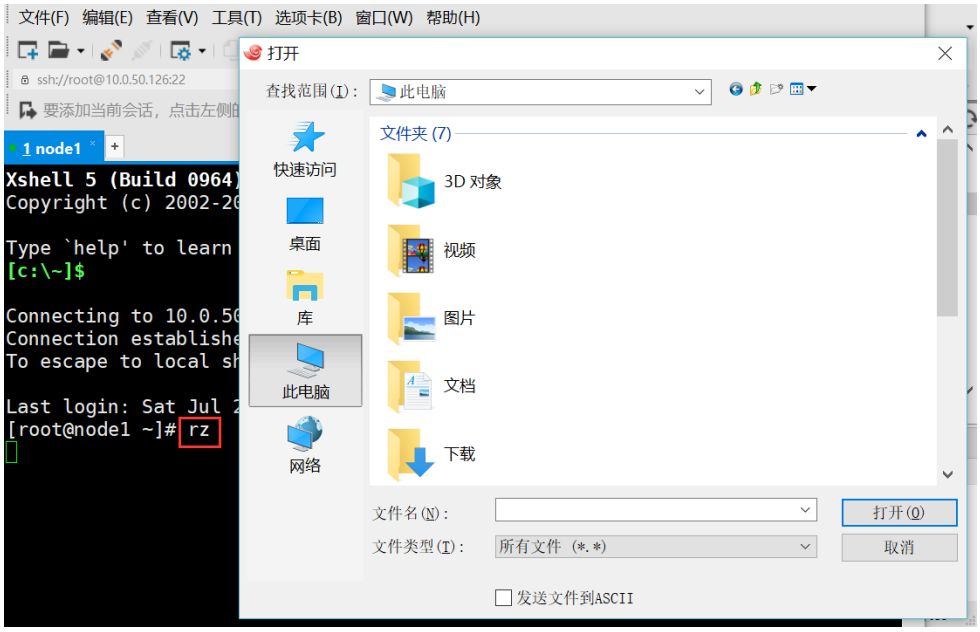


左侧为本机，右侧为高性能集群，可直接拖动进行文件上传和下载。

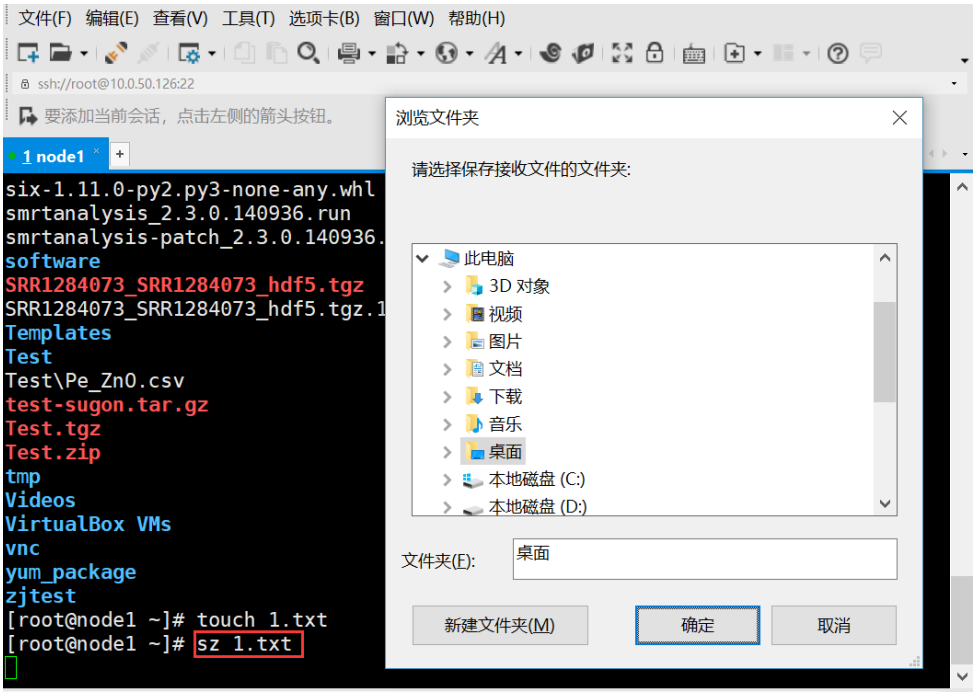


除上述工具外，在命令行中使用 rz 和 sz 命令也可以方便的进行小文件的上传和下载。

$ rz 上传文件



$ sz 下载文件



Linux 操作系统直接使用命令

```
scp  filename   -P2323 test@59.78.189.141:/home/test
```

2.3 编译、安装 OpenMPI 示例

以 OpenMPI 2.1.2 为例：

```
$ tar zxvf openmpi-2.1.2.tar.gz
$ cd openmpi-2.0.1
$ ./configure --prefix=/public/software/mpi/openmpi/intel/2.1.2 --
enable-mpirun-prefix-by-default -without-ucx CC=icc CXX=icpc
FC=ifort F77=ifort
$ make -j 8 && make install
```

设置环境变量脚本：

```
vim /public/software/profile.d/mpi_openmpi-intel-2.1.2.sh
```

```
#!/bin/bash
export MPI_HOME=/public/software/mpi/openmpi/intel/2.1.2
export PATH=${MPI_HOME}/bin:${PATH}
export LD_LIBRARY_PATH=${MPI_HOME}/lib:${LD_LIBRARY_PATH}
export MANPATH=${MPI_HOME}/share/man:${MANPATH}
```

☑ **Tips：**

1. OpenMPI 安装会自动检测编译节点本地可用的通信网络设备（如 Infiniband、Omni-

Path 等），如需支持 InfiniBand 或 Omni-Path 等高速网络，请确保编译 MPI 前该节点已安装 Infiniband 或 OPA 驱动。

2. 执行 OpenMPI 安装目录$MPI_HOME/bin 下的 ompi_info 命令，可查询当前 OpenMPI 配置信息。

2.4 命令行运行程序

2.4.1 运行串行程序

方法一

```
cd /home/your_account/your_workdir
./your_code
```

方法二

```
cd $HOME
vi .bashrc
```

添加

```
export PATH=/home/your_account/your_workdir:$PATH
```

执行命令

```
your_code
```

2.4.2 使用 openmpi 运行并行程序

2.4.2.1 编译 MPI 程序

OpenMPI 提供了 C/C++，Fortran 等语言的 MPI 编译器，如下表所示：

语言类型	MPI 编译器
C	mpicc
C++	mpicxx
Fortran77	mpif77
Fortran90	mpif90

MPI 编译器是对底层编译器的一层包装，通过-show 参数可以查看实际使用的编译器。比如：

```
$ mpicc -show
icc -I/public/software//mpi/openmpi/intel/2.1.2/include -pthread -
Wl,-rpath -Wl,/public/software//mpi/openmpi/intel/2.1.2/lib -Wl,--
enable-new-dtags -L/public/software//mpi/openmpi/intel/2.1.2/lib -
lmpi
```

编译程序示例：

```
$ source /public/software/profile.d/mpi_openmpi-intel-2.1.2.sh
$ mpicc -o hello hello.c
$ mpif90 -o hello hello.f90
```

2.4.2.2 运行 MPI 程序

OpenMPI 使用自带的 OpenRTE 进程管理器，启动命令为 mpirun，基本格式如下：

```
$ mpirun -np N -hostfile <filename> <program>
```

- -np N：运行 N 个进程
- -hostfile：指定计算节点，文件格式如下：

```
node1 slots=8
node2 slots=8
```

slots=8 代表可在该节点上执行 8 个进程，也可将 node1 和 node2 分别写 8 行。hostfile 中的主机名可使用以太网别名（nodeXXX）或 ib 网别名（inodeXXX）或者 IP 地址，但不要将以太网别名和 ib 网别名混用。

Openmpi 运行时会自动检测集群的网络配置，并自动选择 Infiniband 或 OPA 等高速网络进行通信，并不会依赖于 hostfile 中的主机名来选择不同网络。另外，mpirun 后支持多种参数来选择支持的网络，

```
mpirun -mca btl self,sm,openib      //用于 Inifiniband
mpirun -mca mtl psm2                //用于 Omni-Path（OPA）
```

2.4.3 使用 mpich 运行并行程序

2.4.3.1 编译 MPI 程序

MPICH2 编译 C、C++、Fortran77 和 Fortran90 的编译器分别为 mpicc,mpicxx,mpif70 和 mpif90。

语言类型	MPI 编译器
C	mpicc
C++	mpicxx
Fortran77	mpif77
Fortran90	mpif90

编译程序示例：

```
source /public/software/profile.d/mpi_mpich-intel-3.2.sh
mpicc -o hello hello.c
```

```
mpif90 -o hello hello.f90
```

2.4.3.2 运行 MPI 程序

MPICH2 默认使用 hydra 进程管理器，使用 mpirun 启动 MPI 进程，命令格式如下：

```
$ mpirun -f hostfile -n 6 ./program
```

hostfile 文件格式如下：

```
node1:4    #<node name>:<number of cores>
node2:4
```

2.4.4 使用 mvapich2 运行并行程序

2.4.4.1 编译 MPI 程序

```
$ source /public/software/profile.d/mpi_mvapich2-intel-2.3b.sh
$ mpicc -o hello hello.c
$ mpif90 -o hello hello.f90
```

MVAPICH2 编译 C、C++、Fortran77 和 Fortran90 的编译器分别为 mpicc,mpicxx,mpif70 和 mpif90。

2.4.4.2 运行 MPI 程序

MVAPICH2 默认使用 Hydra 进程管理器，启动进程直接使用 mpirun 命令

```
mpirun -f hosts -n 4 [-env ENV value]./program
```

- -f hosts：格式同 MPICH2 相同，<node name>:<proc num>
- -n 4：指定进程数
- -env ENV=value：设置运行环境变量

2.4.5 使用 intelmpi 运行并行程序

2.4.5.1 编译 MPI 程序

IntelMPI 提供了非常完整的 MPI 编译器，既支持 GNU 编译器，也支持 Intel 编译器，具体如下表所示：

MPI 编译器	编译器说明
mpicc，mpigcc	使用 gcc 编译 C 语言
mpicxx，mpigxx	使用 g++编译 C++语言
mpif77	使用 g77 编译 Fortran77 语言
mpif90	使用 gfortran 编译 Fortran90 语言
Mpifc	使用 gfortran 编译 Fortran77/90 语言

Mpiicc	使用 icc 编译 C 语言，推荐使用
Mpiicpc	使用 icpc 编译 C++语言，推荐使用
Mpiifort	使用 ifort 编译 Fortran 语言，推荐使用

可通过 mpiicc -show 来查看具体的编译器信息：

```
$ mpiicc -show

icc -ldl -I/public/software/intel/impi/4.1.0.024/intel64/include -
L/public/software/intel/impi/4.1.0.024/intel64/lib -Xlinker --
enable-new-dtags -Xlinker -rpath -Xlinker
/public/software/intel/impi/4.1.0.024/intel64/lib -Xlinker -rpath -
Xlinker /opt/intel/mpi-rt/4.1 -lmpi -lmpigf -lmpigi -lrt -lpthread
```

编译示例：

```
$ mpiicc -o hello hello.c
```

2.4.5.2 运行 MPI 程序

IntelMPI 也使用 Hydra 进程管理器，进程启动使用 mpirun 命令

```
$ mpirun -np 16 -machinefile hosts -env I_MPI_DEVICE rdma ./program
```

- -machinefile hosts，指定节点列表
- -n 4：指定进程数
- -env ENV=value：设置运行环境变量

另外，IntelMPI 也支持参数来选择高速网络，主要参数如下

```
// Infiniband 高速网络下推荐使用 -DAPL
-genv I_MPI_FABRICS=shm:dapl
// 使用 PSM2 底层通信协议，OPA 网络下推荐使用 -PSM2
-genv I_MPI_FABRICS=shm:tmi
// 使用 libfabric 提供的底层通信协议，OPA 网络下推荐使用（建议较新的驱动版
本下使用）-OFI
-genv I_MPI_FABRICS=shm:ofi
```

2.5 使用作业调度（slurm 命令行模式）运行程序

2.5.1 slurm 的常用命令

命令	功能
sinfo	查看集群队列和节点状态信息
squeue	查看作业和作业步状态
sacct	报告作业/作业步的记账信息。
sstat	报告正在运行的作业/作业步的信息，包含状态采集。

2.5.1.1 sinfo 命令——查看集群队列和节点状态

■ **sinfo命令常用参数**

参数	解释
-a, --all	显示全部队列信息
-n, --nodes=<nodes>	显示节点状态信息
-N, --Node	以每行一个节点方式显示信息，即显示各节点信息。
-p, --partition=<partition>	显示队列信息
-t, --states=<states>	仅显示指定状态的节点状态信息
-r, --responding	仅显示响应的节点信息
-d, --dead	仅显示无响应或已宕机节点状态信息
-R, --list-reasons	显示不响应（down、drained、fail或failing状态）节点的原因
-s	显示摘要信息
-l, --long	显示详细信息

■ **sinfo 常用命令示例**

✓ 查看所有节点信息

```
sinfo
```

```
[root@login01 ~]# sinfo
PARTITION     AVAIL  TIMELIMIT  NODES  STATE NODELIST
normal           up   infinite     17 drain* a3110n[16-19],a3401n16
normal           up   infinite      1  down* b3308r2n5
```

✓ 查看某个分区的节点信息

```
sinfo –p <分区名>
```

```
[root@login01 ~]# sinfo -p normal
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
normal       up   infinite     17 drain* a3110n[16-19],a3401n16
normal       up   infinite      1  down* b3308r2n5
```

✓ 查看某些节点的状态信息

```
sinfo –n <节点名>
```

```
[root@login01 ~]# sinfo -n b3308r2n5
PARTITION     AVAIL  TIMELIMIT  NODES  STATE NODELIST
normal           up   infinite      1  down* b3308r2n5
```

✓ 按照指定状态查看节点信息

```
sinfo –t <节点状态>
```

```
[root@login01 ~]# sinfo -t drain
PARTITION     AVAIL  TIMELIMIT  NODES  STATE NODELIST
normal           up   infinite     17 drain* a3110n[16-19],a3401n16
```

■ **sinfo 命令输出结果示例**

```
adev0: sinfo
PARTITION AVAIL  TIMELIMIT NODES  STATE NODELIST
debug*       up      30:00     2  down* adev[1-2]
debug*       up      30:00     3   idle adev[3-5]
batch        up      30:00     3  down* adev[6,13,15]
batch        up      30:00     3  alloc adev[7-8,14]
batch        up      30:00     4   idle adev[9-12]
```

■ **节点状态说明**

节点状态	描述
idle	节点可用
down/drain	节点不可用
alloc/mix	节点被占用

■ **sinfo 命令输出格式**

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
队列名（分区名）	队列状态（可用性）	默认时间	节点数量	节点状态	节点名列表

2.5.1.2 squeue 命令——查看作业和作业步状态

■ **squeue 命令参数**

参数	解释
-A, --account=<account(s)>	查询指定账号的作业，默认全部账号下的作业
-j, --job=<job(s)>	已逗号分隔指定显示的jobid，默认显示全部
-n, --name=<job_name(s)>	逗号分隔指定的作业名称
-o, --format=<format>	指定显示的信息
-p, --partition=<partition(s)>	逗号分隔指定队列中的作业
-u, --user=<user_name(s)>	逗号分隔指定用户的作业
-s, --steps	显示作业步
-t, --states=<states>	显示特定状态的作业信息
-w, --nodelist=<hostlist>	显示特定节点的作业信息

■ squeue 常用命令示例

✓ 查看所有作业信息

```
squeue
```

```
[root@login01 ~]# squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           4441105    bigmem    test1    user0  R    1:32:52     16 b2106r7n[1-8],b2107r1n[1-8]
           4441229    bigmem    test2    user0  R      49:16      8 b2107r7n[1,5-7],b2108r3n[5-8]
           4431474    normal   py_job    user1  R 1-09:10:51      1 b2107r4n2
           4397345    normal Pleistoc    user2  R 6-04:00:26      6 b2107r7n8,b2108r1n[1-5]
```

✓ 查看某个分区的作业信息

```
squeue –p <分区名>
```

```
[root@login01 ~]# squeue -p bigmem
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           4441105    bigmem    test1    user0  R    1:32:52     16 b2106r7n[1-8],b2107r1n[1-8]
           4441229    bigmem    test2    user0  R      49:16      8 b2107r7n[1,5-7],b2108r3n[5-8]
```

✓ 查看某些节点的作业信息

```
squeue –w <节点名>
```

```
[root@login01 ~]# squeue -w b2107r1n1
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           4441105    bigmem    test1    user0  R    1:32:52     16 b2106r7n[1-8],b2107r1n[1-8]
```

✓ 按照指定状态查看作业信息

```
squeue –t <作业状态>
```

```
[root@login01 ~]# squeue -t R
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           4441105    bigmem    test1    user0  R    1:32:52     16 b2106r7n[1-8],b2107r1n[1-8]
           4441229    bigmem    test2    user0  R      49:16      8 b2107r7n[1,5-7],b2108r3n[5-8]
           4431474    normal   py_job    user1  R 1-09:10:51      1 b2107r4n2
           4397345    normal Pleistoc    user2  R 6-04:00:26      6 b2107r7n8,b2108r1n[1-5]
```

■ squeue 命令查询排队和运行状态的作业

```
JOBID PARTITION  NAME  USER ST  TIME NODES NODELIST(REASON)
65646     batch  chem  mike  R 24:19     2 adev[7-8]
65647     batch   bio  joan  R  0:09     1 adev14
65648     batch  math  phil PD  0:00     6 (Resources)
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
作业号	队列名（分区名）	作业名	用户名	**作业状态**	已运行时间	节点数	**分配给的节点名列表（原因）**

■ 作业状态

作业状态	描述
R	作业运行中
PD	作业排队中
CG	作业完成中

■ 原因

原因	描述
Resources	作业等待其所需的资源可用
Priority	作业所需的队列存在高等级作业或者预留
QOSJobLimit	作业的QOS达到其最大作业计数

2.5.1.3 sacct 命令——报告作业/作业步的记账信息

■ **sacct命令显示运行的或已完成作业或作业步的记账信息。**

参数	解释
--b	显示简要信息，主要包含：jobid、status和exitcode。
-l	显示详细信息
--n	不显示信息头（显示出的信息的第一行，表示个列含义）。
--N <node_list>	显示运行在特定节点的作业记账信息
-E, --endtime=<end_time>	查询在指定时间之前，任何状态的作业.如果通过-s参数指定状态则返回在此时间之前的指定状态的作业，有效格式为： **HH:MM[:SS] [AM|PM]** **MMDD[YY] or MM/DD[/YY] or MM.DD[.YY]** **MM/DD[/YY]-HH:MM[:SS]** **YYYY-MM-DD[THH:MM[:SS]]**
-S, --starttime=<starttime>	在指定时间后，任何状态的作业
--j <jobid>	显示在特定节点数量上运行的作业
-o, --format=<format>	指定显示字段以逗号分隔

■ sacct 常用命令示例

✓ 查看历史作业信息

sacct

```
[user3@login01 ~]$ sacct
       JobID    JobName  Partition    Account  AllocCPUS      State ExitCode
------------ ---------- ---------- ---------- ---------- ---------- --------
1376006      mem_used_+      debug      user3        780    PENDING      0:0
1376007      mem_used_+      debug      user3        780    PENDING      0:0
1376005      mem_used_+      debug      user3        780  COMPLETED      0:0
1376005.bat+ mem_used_+                 user3        780  COMPLETED      0:0
1376005.ext+ mem_used_+                 user3        780  COMPLETED      0:0
```

✓ 查看运行在某个节点上的历史作业信息

sacct -N <节点名>

```
[user3@login01 ~]$ sacct -N b2100r3n2
       JobID    JobName  Partition    Account  AllocCPUS      State ExitCode
------------ ---------- ---------- ---------- ---------- ---------- --------
1376005      mem_used_+      debug      user3        780  COMPLETED      0:0
1376005.bat+ mem_used_+                 user3        780  COMPLETED      0:0
1376005.ext+ mem_used_+                 user3        780  COMPLETED      0:0
```

✓ 查看位于a3310n17节点且在2022-01-22 17:09:16 到2022-01-22 17:09:27这个时间段内的历史作业信息

```
sacct -N a3310n17 -S 2022-01-22T17:09:16 -E 2022-01-22T17:09:27
```

```
[user3@login01 ~]$ sacct -N a3310n17 -S 2022-01-22T17:09:16 -E 2022-01-22T17:09:27
       JobID    JobName  Partition    Account  AllocCPUS      State ExitCode
------------ ---------- ---------- ---------- ---------- ---------- --------
585847             sleep     normal      user3        128  COMPLETED      0:0
585847.batch       batch                 user3         64  COMPLETED      0:0
585847.exte+      extern                 user3        128  COMPLETED      0:0
585847.0       pmi_proxy                 user3          2  COMPLETED      0:0
```

■ **sacct 命令输出格式说明**

```
[user3@login01 ~]$ sacct
       JobID    JobName  Partition    Account  AllocCPUS      State ExitCode
------------ ---------- ---------- ---------- ---------- ---------- --------
1376006      mem_used_+      debug      user3        780    PENDING      0:0
1376007      mem_used_+      debug      user3        780    PENDING      0:0
1376005      mem_used_+      debug      user3        780  COMPLETED      0:0
1376005.bat+ mem_used_+                 user3        780  COMPLETED      0:0
1376005.ext+ mem_used_+                 user3        780  COMPLETED      0:0
```

JobID	JobName	Partition	Account	AllocCPUS	State	ExitCode
作业号	作业名	队列名（分区名）	账户	分配CPU核数	状态	退出码

■ **作业（步）状态**

原因	描述
PENDING	作业排队
COMPLETED	作业（步）正常结束
CANCELLED	作业（步）取消
FAILED	作业（步）失败

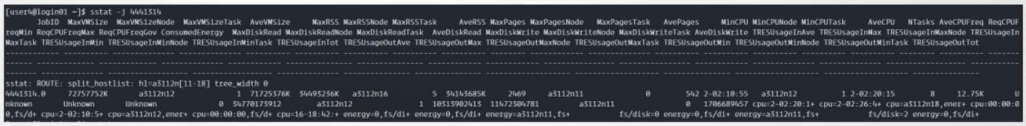
2.5.1.4 sstat 命令——查看正在运行的作业

参数	解释
-a, --allsteps	当没有指定步骤时，显示所有步骤。
-e, --helpformat	打印 –o可以指定的所有参数
-i, --pidformat	列出每个作业步骤运行的pid
-o, --format	格式化输出
-j, --jobs	指定作业jobid

■ sstat 常用命令示例

✓ 查看作业完整信息

```
sstat –j <作业ID>
```



✓ 查看输出格式化字段范围

```
sstat –e
```

```
[user4@login01 ~]$ sstat -e
AveCPU              AveCPUFreq          AveDiskRead         AveDiskWrite
AvePages            AveRSS              AveVMSize           ConsumedEnergy
ConsumedEnergyRaw   JobID               MaxDiskRead         MaxDiskReadNode
MaxDiskReadTask     MaxDiskWrite        MaxDiskWriteNode    MaxDiskWriteTask
MaxPages            MaxPagesNode        MaxPagesTask        MaxRSS
MaxRSSNode          MaxRSSTask          MaxVMSize           MaxVMSizeNode
MaxVMSizeTask       MinCPU              MinCPUNode          MinCPUTask
Nodelist            NTasks              Pids                ReqCPUFreq
ReqCPUFreqMin       ReqCPUFreqMax       ReqCPUFreqGov       TRESUsageInAve
TRESUsageInMax      TRESUsageInMaxNode  TRESUsageInMaxTask  TRESUsageInMin
```

✓ 查看作业内存消耗等常见信息

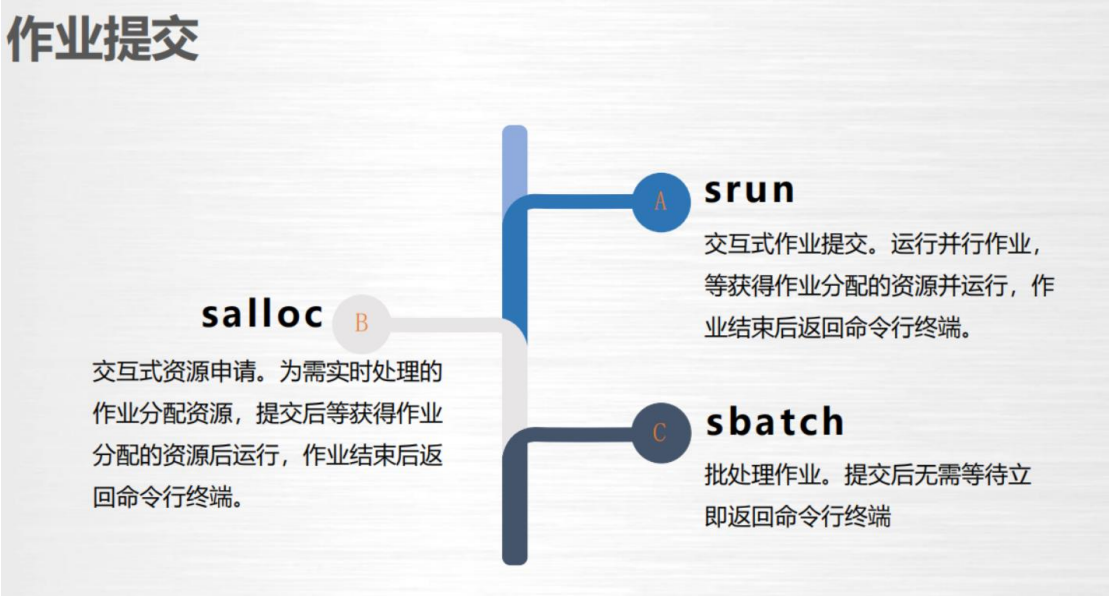
```
sstat –j <作业ID>-o jobid%16,nodelist,ntasks,averss,maxrss,avevmsize,maxvmsize,pids
```

```
[user4@login01 ~]$ sstat -j 4441314 -o jobid%16,nodelist,ntasks,averss,maxrss,avevmsize,maxvmsize,pids
          JobID             Nodelist   NTasks     AveRSS     MaxRSS  AveVMSize  MaxVMSize                Pids
---------------- -------------------- -------- ---------- ---------- ---------- ---------- --------------------
sstat: ROUTE: split_hostlist: hl=a3112n[11-18] tree_width 0
       4441314.0        a3112n[11-18]        8 34145365.+  34493500K  71725376K  72757752K 10796,10800,10801,1+
```

```
Tasrs: 929 total,  33 runneng, 896 slsspeng,   0 stoppsd,   0 zombes
%Cpu(s): 49.3 us,  0.7 sy,  0.0 ne, 49.8 ed,  0.0 wa,  0.0 he,  0.1 se,  0.0
KeB Msm : 26168352+total, 20240814+urss, 45056840 ussd, 14218540 buuu/4a4hs
KeB Swap: 16364540 total, 16363772 urss,      768 ussd. 21053998+avael Msm

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
10805 user4     20   0 2297064   1.1g  34488 R 105.9  0.4 106:52.78 44sm.sxs
10808 user4     20   0 2291516   1.0g  32916 R 105.9  0.4 107:00.76 44sm.sxs
10815 user4     20   0 2185332 993140  32596 R 105.9  0.4 106:52.93 44sm.sxs
10816 user4     20   0 2174968 975304  34748 R 105.9  0.4 106:58.79 44sm.sxs
10826 user4     20   0 2181504 984864  31320 R 105.9  0.4 106:54.16 44sm.sxs
10800 user4     20   0 2275648   1.1g 125896 R 100.0  0.5 106:29.39 44sm.sxs
10801 user4     20   0 2268852   1.0g  33716 R 100.0  0.4 106:57.87 44sm.sxs
```

2.5.2 作业提交命令



通用参数

■ 适用于srun、salloc和sbatch命令

指定参数	参数含义
-J , --job-name	指定作业名称
-p, --partition	指定队列资源
-N, --nodes=<number>	指定节点数量
-n, --ntasks=<number>	指定作业任务数量
--ntasks-per-node=<number>	**指定每节点任务数，最大为节点物理核心数，默认1**
--cpus-per-task=<number>	**指定每任务CPU核心数对应多线程场景，需配合OMP_NUM_THREADS变量，默认1**
--mem=xx	预留申请内存容量，单位MB、GB
--gres=xxx:<number>	申请加速卡数，最大为节点加速卡数量
-w, --nodelist=<node name list>	指定申请的节点，格式可以为node1，node2 或者node[1-10]
-x, --exclude=<node name list>	排除指定的节点，格式可以为node1，node2 或者node[1-10]
-t, --time=DD-HH:MM	指定作业运行时间，例如7-08:00代表7天8小时
--exclusive	设置节点独占

CPU 资源分配规则

✓ 单线程时，作业分配的总进程数等于分配的CPU核心数

ntasks-per-node * N<= N * 单节点CPU核心数
n <= N *单节点CPU核心数

```
[zlei@login01 ~]$ srun -p high -N 2 -n 128 --ntasks-per-node=64 hostname
srun: job 4478388 queued and waiting for resources
srun: job 4478388 has been allocated resources
srun: ROUTE: split_hostlist: hl=e2113r4n[3-4] tree_width 0
e2113r4n3
```

✓ 多线程时，作业分配的进程数不等于分配的CPU核心数，还要考虑线程的核心占用情况，且OMP_NUM_THREADS应等于cpus-per-task。

ntasks-per-node * cpus-per-task <= 单节点CPU核心数
n * cpus-per-task <= N *单节点CPU核心数

```
[zlei@login01 ~]$ srun -p high -N 2 -n 64 --ntasks-per-node=32 --cpus-per-task=2 hostname
srun: job 4478390 queued and waiting for resources
srun: job 4478390 has been allocated resources
srun: ROUTE: split_hostlist: hl=e2113r4n[3-4] tree_width 0
e2113r4n3
e2113r4n3
```

2.5.2.1 srun 命令交互式作业提交

■ **srun 命令交互式作业提交示例**

✓ 指定分区、节点数量和任务数提交作业

srun -p <分区> -N <节点数> -n <总任务数> <作业脚本或者命令>

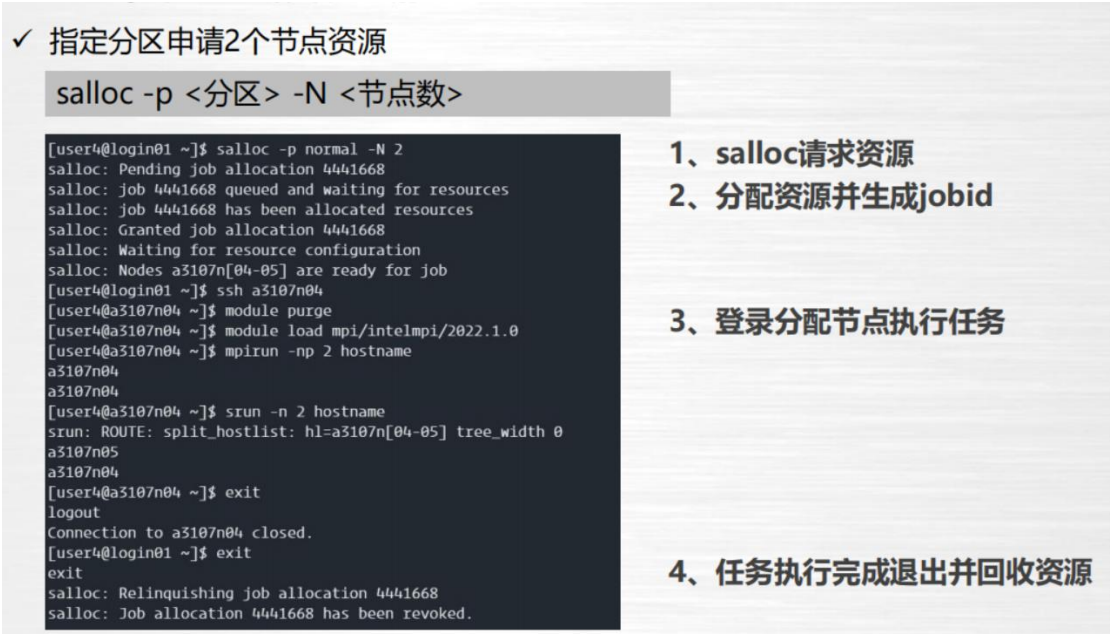
```
[user4@login01 ~]$ srun -p normal -N 2 -n 4 hostname
a3107n04
a3107n04
a3107n04
a3107n05
```

✓ 指定分区、节点数量和任务数提交作业

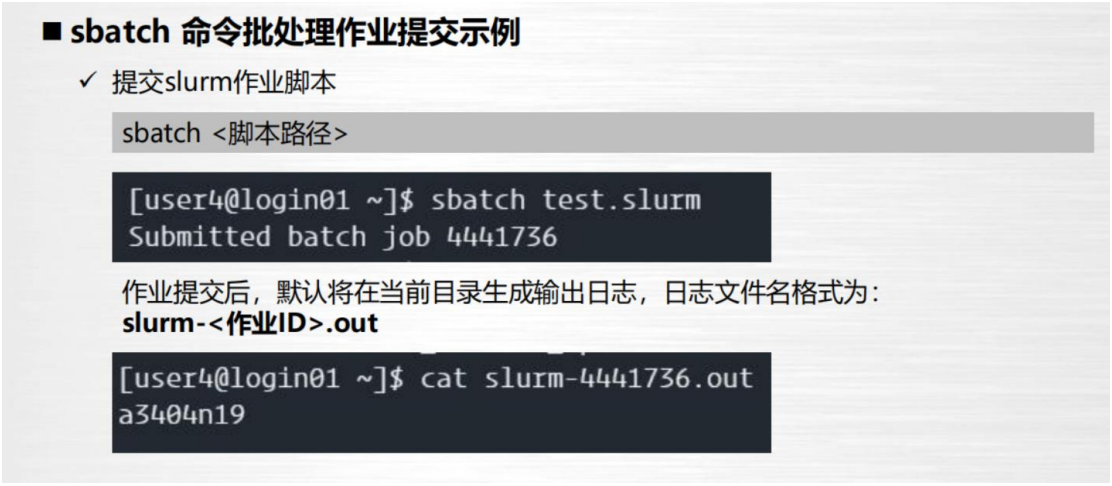
srun -p <分区> -w <节点> -n <总任务数> <作业脚本或者命令>

```
[user4@login01 ~]$ srun -p normal -w a3107n04 -n 2 hostname
a3107n04
a3107n04
a3107n04
a3107n04
```

2.5.2.2 salloc 命令交互式资源申请



2.5.2.3 sbatch 命令批处理提交作业



2.5.2.4 作业提交环境变量

■ slurm作业可以通过以下变量获取作业信息

Slurm环境变量	功能
$SLURM_JOB_ID	作业ID
$SLURM_JOB_NAME	作业名
$SLURM_JOB_PARTITION	队列的名称
$SLURM_NTASKS	任务（进程）总数
$SLURM_NTASKS_PER_NODE	每个节点请求的任务数
$SLURM_JOB_NUM_NODES	节点数
$SLURM_JOB_NODELIST	节点列表
$SLURM_LOCALID	作业进程的节点本地任务ID
$SLURM_ARRAY_TASK_ID	作业组中的任务ID
$SLURM_SUBMIT_DIR	工作目录
$SLURM_SUBMIT_HOST	提交作业的主机名

```
[user4@login03 ~]$ cat job3.slurm
#!/bin/bash
#SBATCH -J echo_env
#SBATCH -p normal
#SBATCH -N 2
#SBATCH -n 2
#SBATCH --ntasks-per-node=1

echo "Allocate Nodelist is ${SLURM_JOB_NODELIST}"

sleep 10

[user4@login03 ~]$ sbatch job3.slurm
Submitted batch job 4474470
[user4@login03 ~]$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           4474470    normal echo_env     user4  R       0:01      2 a3107n[16-17]
[user4@login03 ~]$ cat slurm-4474470.out
Allocate Nodelist is a3107n[16-17]
[user4@login03 ~]$
```

2.5.2.5 作业提交 PBS VS SLURM

功能	SLURM	PBS
任务名称	#SBATCH -J name	#PBS -N name
指定队列/分区	#SBATCH -p cpu	#PBS -q cpu
指定 QoS	#SBATCH --qos=debug	#PBS --qos=debug，需调度器支持
最长运行时间	#SBATCH -t 5:00	#PBS -l walltime=5:00
指定节点数量	#SBATCH -N 1	#PBS -l nodes=1
指定 CPU 核心	#SBATCH --cpus-per-task=4	#PBS -l ppn=4
指定加速卡	#SBATCH --gres=gpu:1	不支持
作业数组	#SBATCH -a 0-2	#PBS -t 0-2
输出文件	#SBATCH -o test.out	#PBS -o test.out
提交任务脚本	sbatch run.slurm	qsub run.pbs
查看任务状态	squeue	qstat
取消任务	scancel 1234	qdel 1234
交互式任务	salloc，手动切换	qsub -I，自动切换
指定特定节点	#SBATCH --nodelist=comput1	qsub -l nodes=comput1

2.5.2.6 取消作业

■ **scancel 命令用于取消挂起或正在运行的作业或作业步骤。**

指定参数	参数含义
-n, --name=<job_name>	指定作业名
-p, --partition=<partition>	指定分区名
-t, --state=<states>	指定作业状态
-u, --user=<user_name>	指定用户名
-w, --nodelist=<hostlist>	指定节点

■ **scancel 命令用于取消挂起或正在运行的作业或作业步骤。**

✓ 取消作业

```
scancel <作业ID>
```

```
[user4@login01 ~]$ squeue -j 4445358
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           4445358    normal     test    user4 PD       0:00      1 (Nodes required
[user4@login01 ~]$ scancel 4445358
[user4@login01 ~]$ squeue -j 4445358
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
```

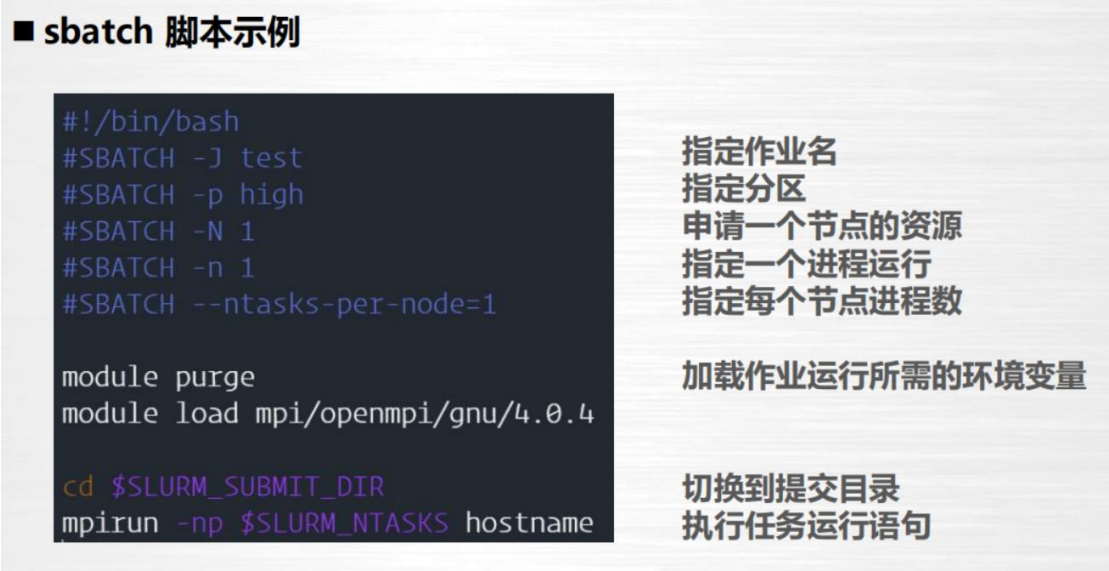
■ **处于完成或失败状态的job如何重新排队**

Slurm 支持重新安排处于完成或失败状态的作业。可以使用命令：

```
scontrol requeue <作业ID>
```

然后，该作业将被重新排队，回到 **PENDING** 状态

2.5.3 作业调度系统使用举例



注意事项

1、非 root 用户只能查看、删除自己提交的作业

2、在提交作业时尽可能的根据自己的使用的机器数估算内存，把其写进作业提交脚本里。

3、集群限制 root 用户提交任务

4、检查 SLURM 脚本是否正确，任务投放到对应的队列上

5、在 7 期集群的公共访问目录/public/share 下是通用的 SLURM 调度系统的脚本模板，可以参照。可以通过登录节点上执行 gpus，查看各个 gpu 节点上未被申请使用的 gpu 卡数量。

6、集群所有计算节点（包括 gpu 节点）限制用户直接登录访问，用户必须通过调度系统提交任务运行

2.6 SLURM 作业提交步骤

- STEP1　准备：编写描述改作业的脚本，包括作业名，需要的资源等。
- STEP2　提交：使用 sbatch 命令将该作业提交给 SLURM 服务器
- STEP3　排队：服务器将该任务排入适当的队列
- STEP4　调度：服务器检查各工作节点的状态是否符合该作业的要求，并进行调度。
- STEP5　执行：当条件满足时，作业被发给相应的执行服务器执行。程序运行时执行服务器会收集程序的标准输出和标准错误流，等程序结束时，将这些信息返回给用户。
- STEP6　查询和调整：当作业在运行时，用户可以使用 squeue 进行状态查询。用户发现作业提交错误时，可以使用 scancel 删除正在运行的作业。

■ STEP7 查看结果：使用文本编辑软件 vi 或者系统命令 cat, less 等查看输出及错误信息显示。

3 软件安装及使用规范

1. 7、8 期集群系统默认安装了部分公共软件环境，位置在 /public/software 下，主要为 Intel 编译器、MPI 及数学库。
2. 用户自行运行的软件，建议安装于自己属主目录下，如果有特殊需求，可以申请安装于 /public/software 等公用目录下；如果该软件使用人员较多，软件使用有通用性，推荐安装于公用目录/public/software 下，减少对属主目录的空间占用。
3. 用户使用时，应注意自己账号的环境变量，修改自己账号属主目录下的.bashrc 文件，或者通过集群的 module 环境变量管理工具；设置正确的用户环境变量。
4. 用户的任务脚本中，应尽量使用相应计算软件的绝对路径，指明应用软件所在位置。
5. 严禁未经许可擅自修改公用软件。

用户环境变量配置举例：
vi ~/.bashrc
######## intel ########
source /public/software/compiler/intel/intel-compiler-2017.5.239/bin/compilervars.sh intel64

source /public/software/mpi/intelmpi/2017.4.239/bin64/mpivars.sh

或者使用module命令：
module load compiler/intel/intel-compiler-2017.5.239

module load mpi/intelmpi/2017.4.239

######说明####

当前7、8期安装的公用软件环境，都写好了对应的环境变量文件，文件都位于目录/public/software/profile.d下。用户使用的使用可以通过source 对应的文件来生效相应的环境。

4 关机和启动

1. 遇有紧急情况和计划性维护工作时，集群系统将停机关机，请各用户接获通知后，应尽量注意保证提交的计算任务能够在预定的停、关机时间前结束。

2. 系统重新启动后，具体集群管理维护会及时启动相应服务，并进行集群环境检查。

   紧急关机并重新启动后，请使用人员各自应检查关机前计算任务的状态和结果，注意确保计算结果的无误和完整，并注意文件和目录是否出现的访问异常情况。

5 集群管理人员联系方式

1. 电话： 13918195801
2. 邮件：

   wxhu@cc.ecnu.edu.cn

   kyu@cc.ecnu.edu.cn
3. 微信号： 18912086789

6 使用问题反馈

如果大家在集群使用中，作业任务出现异常、故障等问题，需要帮助时，需要尽量多提供一些信息，以方便管理人员更好的测试、排查和定位问题。

需要提供的信息：

哪一期的集群 ， 用户账号，问题作业的 ID 号，作业的运行目录，作业的脚本，以及一些错误的输出信息等。